

COMSATS University Islamabad

Lahore Campus — Department of Computer Science



Theory Assignment **OBJECT ORIENTED PROGRAMMING**

Object-Oriented Analysis of a Smart Campus Allocation System

Submitted By:

Name: Qirat Hafsa

Registration No:

SP26-BAI-041

Submitted To:

Dr. Muhammad Shahid Bhatti

Assigned Condition

Privacy First (condition 1)

Submission Date:

25-09-2026

Task 1 – Stakeholder and Requirement Analysis

Facts vs. assumptions

Fact (given): the system is a Smart Campus Allocation System that lets campus users reserve spaces, and the assigned condition requires that ordinary users can see whether a space is available but must not see the identity or purpose of another person's reservation.

Assumptions (introduced by me, since a detailed case narrative was not reproduced beyond the condition above): the campus has bookable classrooms, labs and halls; users include students, faculty and a facilities/admin office; a reservation always has a fixed date, time-slot and space; an academic timetable already blocks some slots before ad-hoc booking opens.

Stakeholders and needs

- **Student** – needs to quickly find and book a free room for a group project or study session, without seeing who else is using nearby rooms.
- **Faculty member** – needs to reserve rooms for extra classes or office hours and expects these bookings to be respected over informal student requests.
- **Facilities/Admin staff** – needs full visibility of a reservation (identity, purpose, contact) to resolve disputes, schedule maintenance and enforce policy.
- **System Administrator (IT)** – needs every booking and access decision logged for audit, while the privacy rule is still enforced for ordinary users.

Genuine requirement conflicts

- **Transparency vs. privacy:** admin staff want full booking detail visible; the assigned condition forbids showing identity/purpose to ordinary users. Decision needed before design: define two distinct “views” of a reservation (masked / full) and decide which roles receive which view.
- **Fairness vs. priority:** a faculty class-linked booking and a first-come student request can target the same room and slot. Decision needed: whether role-based priority is automatic or goes through an approval step, and whether it can override an already-confirmed booking.
- **Flexibility vs. predictability:** users want to cancel or change bookings freely; admin needs a stable schedule for maintenance planning. Decision needed: a cut-off window before which cancellation is free and after which the slot is still logged as briefly unavailable.

Task 2 – Object-Oriented Conceptual Model

Classes

- **User (abstract)** – represents any campus account. Attributes: `userId`, `name`. Operations: `login()`, `viewAvailability()`.
- **Student (extends User)** – a learner who books rooms for personal/group use. Attributes: `studentId`, `programme`. Operation: `requestReservation()`.
- **Faculty (extends User)** – an instructor who books rooms for teaching. Attributes: `employeeId`, `department`. Operation: `requestReservation()`.

- **AdminStaff (extends User)** – facilities/admin employee. Attributes: office. Operations: resolveConflict(), viewFullDetails().
- **Space** – a bookable room/hall. Attributes: spaceId, capacity. Operations: checkAvailability(), block().
- **TimeSlot** – a value object for a date/time span. Attributes: startTime, endTime. Operation: overlaps(other).
- **Reservation** – a single booking request. Attributes: reservationId, purpose (hidden), status. Operations: confirm(), cancel().
- **AllocationManager** – coordinates booking and conflict logic. Operations: allocateSpace(), detectConflict().
- **PrivacyFilter** – decides how much reservation detail a viewer may see. Attributes: none (stateless policy). Operation: filterView(reservation, viewerRole).
- **Notification** – a message sent to an affected user. Attributes: message, recipient. Operation: send().

Class diagram

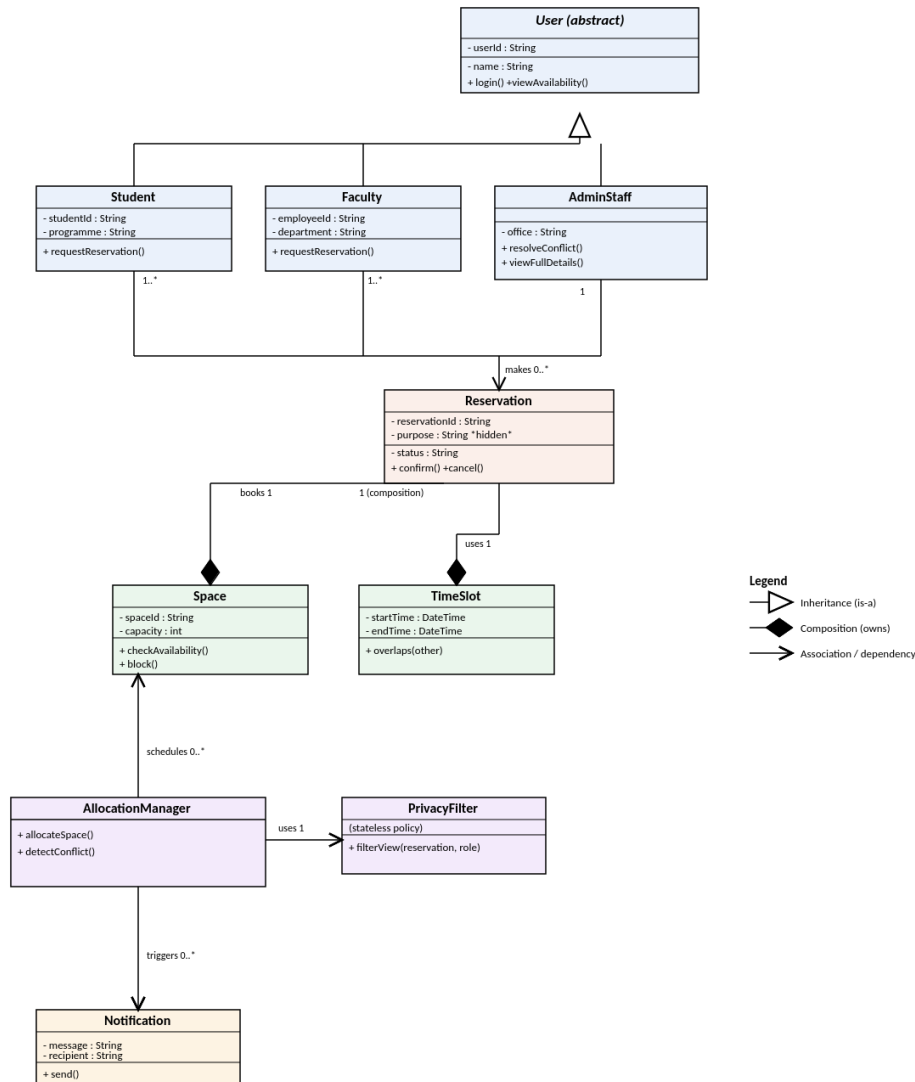


Figure 1. UML class diagram of the Smart Campus Allocation System.

Separation of responsibilities

No single class does everything. Space only knows its own physical state and answers availability queries; Reservation only knows its own life-cycle; AllocationManager coordinates across many reservations and spaces but delegates the actual privacy decision to PrivacyFilter, and delegates delivery to Notification. This keeps conflict detection, privacy policy, and messaging as three independently changeable responsibilities instead of one controller class that stores everyone's data and makes every decision.

Task 3 – Abstraction, Encapsulation and Invariants

Reservation

Abstracts: the real-world act of claiming a specific space for a specific time period.

Hidden: requester identity and purpose (per the privacy-first condition). Changed only via `confirm()`, `cancel()` and `reassignSpace()` – never by writing fields directly, so status and privacy rules are always applied together.

Space

Abstracts: a physical room/hall with a fixed capacity and a current schedule.

Hidden: the full internal list of bookings and any maintenance flag. Exposed only through `checkAvailability()`, which returns free/busy, not who booked it. Changed via `block()/unblock()`, callable only by `AdminStaff` or `AllocationManager`.

User (and its subclasses)

Abstracts: a member of the campus community with an identity and a booking history.

Hidden: personal profile and reservation history, visible to the owner and to `AdminStaff` only, never to another ordinary user. Changed via `register()` and `updateProfile()`.

Invariants

- A Reservation's TimeSlot must never overlap another Confirmed reservation for the same Space (no double-booking).
- An ordinary User (Student/Faculty) viewing another user's reservation may see only status and time-slot – never requester identity or purpose.
- A Reservation may move to Confirmed only from Requested, never directly from Cancelled or Rejected.
- A Space marked blocked (maintenance/closure) cannot accept a new Confirmed reservation until unblocked.
- Every Reservation must reference exactly one Space and one TimeSlot – an orphan reservation is not allowed.

Because these rules are enforced inside class methods rather than left to external code, encapsulation prevents invalid states such as two confirmed bookings for one room, or a leaked identity, that would be possible if callers could edit fields directly.

Task 4 – Alternative Designs and Decision

Campus users can hold more than one role over time – e.g. a Student who becomes a part-time lab assistant, or a Faculty member temporarily given admin rights during an emergency.

Option A – Inheritance

Student, Faculty and AdminStaff are subclasses of User. A person with two roles needs either a new combined subclass (e.g. StudentAssistant) or multiple inheritance, which most languages restrict.

Option B – Composition / role objects

User holds a collection of Role objects (StudentRole, FacultyRole, AdminRole). Each Role carries its own permissions, and roles can be attached to or removed from a User at run time.

Comparison

Criterion	Inheritance	Composition / roles
Extensibility	New combination $\Rightarrow$ new subclass; hierarchy grows fast	New Role class added independently, attached to any User
Coupling / duplicate behaviour	Shared behaviour often re-declared on siblings	Shared behaviour stays in User; role logic isolated
Multiple / changing roles at run time	Fixed at object creation; role change $\Rightarrow$ replace object	Roles added/removed on the same User object at any time

Decision

I choose composition with role objects. Campus roles change mid-semester (a student becoming a TA, a faculty member gaining temporary admin rights during an emergency), and the privacy-first condition needs PrivacyFilter to ask “does this viewer currently hold an Admin-level role” at run time – a natural check against a Role collection, not a fixed class type.

Task 5 – Behaviour and Edge Cases

Reservation life cycle (state table)

Current state	Event	Next state
–	requestReservation()	Requested
Requested	AllocationManager confirms (no conflict)	Confirmed
Requested	conflict detected	Rejected
Requested / Confirmed	requester cancels	Cancelled
Confirmed	Space blocked (maintenance) or building closure	Relocated
Relocated	alternative space+slot accepted	Confirmed
Relocated	no alternative available	Cancelled
Confirmed	time-slot elapses normally	Completed

Edge cases

- **Two simultaneous requests for the same space/time:** AllocationManager.detectConflict() checks overlapping reservations for that Space before confirming either one; only one transitions to Confirmed. Enforced by AllocationManager, the only class with visibility across all reservations for a space.
- **Maintenance block after confirmation:** AdminStaff calls Space.block(); AllocationManager finds the affected Confirmed reservations, moves each to Relocated, and tries to reassign a Space+TimeSlot; Notification informs each affected user. Enforced jointly by Space (announces the status change) and AllocationManager (owns reallocation).

- **Emergency cancellation/relocation (building closure):** the status change cascades through AllocationManager to every Confirmed reservation in that building – relocate where possible, else Cancelled – and Notification is sent to each requester. PrivacyFilter still ensures a third party's notification ("Room X unavailable") never reveals another user's identity or purpose. Enforced by AllocationManager (bulk rule), PrivacyFilter (keeps messages privacy-safe) and Notification (delivery).

Task 6 – Independent Exploration: Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is an access-control approach in which permissions are assigned to roles rather than to individual users, and a user is simply granted membership in one or more roles [1]. Instead of checking "is this a Faculty object", the system checks "does this user currently hold the Faculty or AdminStaff role", which matches the run-time Role-object design chosen in Task 4. In this system, RBAC gives a precise mechanism for the privacy-first condition: PrivacyFilter (Task 2) decides how much of a Reservation to reveal by consulting the viewer's active roles instead of hard-coded identity checks. An ordinary Student or Faculty role sees only availability and time-slot; only the AdminStaff role is granted permission to view full reservation detail, matching the least-privilege principle central to RBAC [2]. This keeps privacy enforcement centralised in one policy class and auditable, rather than scattered as ad-hoc identity checks throughout the code.

Generative AI Disclosure

Generative AI (Claude, by Anthropic) was used to help organise this submission, draft prose, and format the class diagram. All stakeholder claims, class decisions, invariants, state transitions and the design choice in Task 4 were reviewed and are presented here as my own analysis. I remain fully responsible for the accuracy of every claim and the correctness of every design decision in this submission.

References

- [1] D. F. Ferraiolo and D. R. Kuhn, "Role-based access controls," in Proc. 15th National Computer Security Conf., Baltimore, MD, USA, 1992, pp. 554–563.
- [2] D. F. Ferraiolo, J. F. Barkley, and D. R. Kuhn, "A role-based access control model and reference implementation within a corporate intranet," ACM Transactions on Information and System Security, vol. 2, no. 1, pp. 34–64, 1999. doi: 10.1145/300830.300834.